CSRF y acciones no deseadas

Diapositivas de apoyo: Una acción que nadie pidió y Una cookie no prueba intención. Ensayar a 125-145 palabras por minuto. Mantener clara la diferencia con XSS, que acaba de explicarse.

Estar conectado no demuestra una decisión

Mi parte comienza con una pregunta: si el servidor recibe una petición con nuestra sesión, ¿significa que quisimos realizar esa acción? A primera vista parece que sí, porque somos nosotros quienes iniciamos sesión. Pero el navegador puede enviar ciertas solicitudes y adjuntar cookies automáticamente bajo sus reglas. Esa comodidad permite usar una página sin escribir la contraseña una y otra vez. Al mismo tiempo, obliga al servidor a distinguir entre reconocer una sesión y comprobar que una acción corresponde al flujo que la aplicación espera.

CSRF significa falsificación de solicitudes entre sitios. El nombre técnico describe una situación en la que un sitio externo induce al navegador de una persona autenticada a enviar una acción a otro sitio. Si el segundo sitio confía solamente en la cookie y no tiene una protección adecuada, podría aceptarla. El atacante aprovecha la sesión que la víctima ya tiene. No necesita conocer necesariamente la contraseña para intentar provocar esa acción. El resultado depende de las reglas del navegador, de las cookies y de las validaciones del servidor.

Imaginemos que tenemos abierta una plataforma de cursos. En ella podemos cambiar nuestro nombre visible. Después visitamos otra página en otra pestaña. Esa página intenta provocar una solicitud hacia la plataforma para cambiar ese dato. Si la solicitud puede incluir la credencial de sesión y el servidor no exige nada más, podría procesarse como una acción nuestra. El ejemplo es ficticio y simplificado, pero muestra el punto esencial: la plataforma reconoce una cuenta sin comprobar suficientemente la intención asociada a esa operación.

Cómo se forma la situación

Podemos separar el funcionamiento en tres condiciones. La primera es que exista una sesión o una credencial que el navegador adjunte automáticamente en la solicitud relevante. La segunda es que otra página pueda inducir una petición con una forma aceptada por la aplicación. La tercera es que el servidor no valide una defensa adecuada antes de modificar el estado. Si falta alguna condición, el escenario cambia. Por eso no es correcto afirmar que cualquier formulario de cualquier sitio puede ejecutar cualquier operación en nombre de cualquiera.

Modificar el estado significa cambiar algo persistente o significativo: guardar una configuración, actualizar un perfil o confirmar una operación. Consultar una página y modificar información no deberían confundirse. Las solicitudes GET se utilizan para obtener recursos y no deberían ejecutar cambios de estado. Si un simple enlace produce una modificación, se amplían los lugares desde donde esa acción podría desencadenarse. Usar POST para cambios ayuda a respetar el diseño de HTTP, pero POST por sí solo no representa una protección completa contra CSRF.

Los riesgos dependen de las operaciones disponibles. En un sistema podría cambiarse un correo de contacto; en otro, una preferencia o una acción administrativa. El ataque no necesita devolver la respuesta al sitio externo para provocar un daño. Puede ser suficiente con que la acción se acepte. Esta es una diferencia útil respecto a la idea de robar información: algunas amenazas buscan que algo ocurra, aunque quien las provoca no pueda leer directamente la respuesta. El servidor debe proteger la operación, no solamente la visualización de los datos.

Libreto de la comparación: La misma cuenta. Otra intención.

En la izquierda estamos dentro del perfil de Ana. Ella solicita cambiar su correo a ana.nuevo y utiliza el formulario del sitio. El servidor recibe el flujo esperado, con su comprobación CSRF, y guarda el cambio elegido por la propia persona.

En la derecha, Ana sigue conectada al campus, pero visita una página de premios. El botón dice Ver mi cupón. Detrás, esa página prepara otra acción: enviar a la cuenta de Ana un correo que pertenece a un tercero. Cuando se representa el clic, observemos el correo guardado. El sitio vulnerable aceptó una modificación que Ana no pidió porque confió únicamente en la cookie.

Esta comparación supone que el navegador permite adjuntar la cookie. SameSite puede impedir determinados envíos; no afirmamos que cualquier página externa siempre pueda hacerlo. La diferencia con XSS es que aquí mostramos una solicitud inducida desde otra página, no un script publicado como comentario.

La diferencia con XSS

XSS y CSRF pueden afectar al mismo usuario, pero no describen la misma causa. En XSS, contenido no confiable se ejecuta en el contexto de una página. En CSRF, una página externa induce una petición que aprovecha credenciales enviadas automáticamente. Una forma de recordarlo es pensar en ejecutar dentro del sitio frente a solicitar desde otro sitio. Esta comparación simplifica la explicación sin sustituir todos los detalles técnicos. También nos ayuda a entender por qué una defensa aplicada a un problema no necesariamente resuelve el otro.

Token CSRF: una comprobación adicional

La defensa habitual para formularios con sesión es un token CSRF validado por el servidor. Un token es un valor que la aplicación genera y comprueba como parte del flujo. El formulario legítimo lo incluye y el servidor rechaza operaciones que no presentan una prueba válida. Podemos compararlo con un comprobante asociado al trámite, además de la credencial de identidad. La analogía tiene límites, pero ayuda a separar dos preguntas: qué sesión participa y si la solicitud cumple la comprobación que espera la aplicación.

No necesitamos diseñar nuestro propio mecanismo durante la exposición. En Flask podemos usar Flask-WTF y activar su protección CSRF. La biblioteca ayuda a generar y validar los tokens dentro de la aplicación. Es importante que el control ocurra en el servidor. Un campo oculto es invisible en la interfaz normal, pero cualquier persona puede observar y modificar su propio formulario con herramientas del navegador. Su protección no depende de estar escondido; depende de la generación y de la validación realizadas correctamente.

En el laboratorio se enviará primero un formulario válido. Después se quitará el campo del token mediante las herramientas de desarrollo y se volverá a enviar. La aplicación responderá con un error de solicitud, porque falta la prueba esperada. Esto demuestra el rechazo de una solicitud sin token. Debemos decirlo con precisión: no estamos reproduciendo un ataque completo desde otro origen, sino verificando una defensa específica. Esa honestidad permite que el público comprenda exactamente qué evidencia está viendo y evita atribuirle al ejemplo más alcance del que tiene.

Otras comprobaciones útiles

SameSite es un atributo de cookies que influye en cuándo se envían entre sitios. Las opciones Lax y Strict restringen diferentes flujos. El comportamiento depende también del tipo de navegación y de la solicitud. No hace falta memorizar todos los casos para recordar la recomendación: configurarlo de forma explícita y considerarlo una capa adicional. No debemos concluir que todas las cookies siempre acompañan cualquier petición externa. Precisamente por esas reglas, al explicar el ataque utilizamos la condición si el navegador permite enviarlas.

Para cerrar, pensemos de nuevo en la plataforma de cursos. Una cookie válida permite reconocer una sesión; un token y otras comprobaciones ayudan a controlar la petición; la autorización confirma que esa cuenta puede realizar la acción concreta. Son preguntas diferentes. Si las contestamos de forma explícita, dejamos menos decisiones apoyadas en suposiciones. Mi compañero continuará con el cuidado de las sesiones: cómo se utilizan esas credenciales temporales, qué pasa si otra persona las obtiene y por qué su duración también forma parte de la seguridad.

Apoyo para comprender y ensayar

CSRF se puede pronunciar letra por letra. Estado significa información o situación que cambia por una operación. Token no significa contraseña y no debe explicarse como un campo secreto por estar oculto. Si preguntan si usar POST basta, responder que no: también hace falta la validación. Hacer una pausa antes de comparar XSS con CSRF y otra antes de presentar el token. No añadir una segunda demostración en vivo en esta parte.

Indicaciones para presentar esta parte

Abrir la comparación después de Una acción que nadie pidió. Señalar que las direcciones de las ventanas son diferentes. Esperar el cambio de correo antes de explicar el resultado. No decir que el atacante conoce la contraseña ni que la cookie se envía siempre. Continuar a la prevención de CSRF sin interactuar con las ventanas.

Diccionario de términos

CSRF: Petición inducida desde otro sitio que intenta aprovechar credenciales enviadas automáticamente.

Token CSRF: Valor que el servidor genera y valida para proteger solicitudes que cambian información.

GET: Método HTTP para obtener recursos; no debería modificar datos.

POST: Método utilizado para enviar datos; no protege por sí solo contra CSRF.

SameSite: Atributo de cookie que limita determinados envíos entre sitios.

Origen: Combinación de esquema, host y puerto que delimita una parte de la confianza del navegador.

Origin y Referer: Encabezados que pueden ayudar al servidor a verificar de dónde procede una solicitud.

Flask-WTF: Biblioteca que incluye protección CSRF para aplicaciones Flask.

HTTP 400: Respuesta que indica que la solicitud no es válida; aquí permite observar el rechazo sin token.